



# Natural Language Processing ( Spring-2026 )

## Assignment-1

**Instructor Dr. Hajra Waheed**

### Teacher Assistant

- **Abdur Rehman:** [abdurrehman.ta2025@lhr.nu.edu.pk](mailto:abdurrehman.ta2025@lhr.nu.edu.pk)
- For **assignment & Viva-related queries**, please contact the TA directly via email.

### Submission Guidelines:

- Submit your assignment on Google Classroom in the format "20XX.ipynb".
- **The deadline is Feb 20, 2026, at 11:59 PM. No extensions will be granted.**

### Declarations:

- Late submissions will incur penalties: 25% deduction on the first day, 50% on the second day, and zero marks thereafter.
- **Plagiarism will result in zero marks for the assignment.**
- This is an individual assignment; collaboration or group work is strictly prohibited.
- Please ensure that you submit your own original work.

### VIVA Policy:

- A VIVA (oral examination) will be conducted to assess your understanding of the assignment.
- The VIVA will be scheduled separately, and you will be notified of the date and time.
- Failure to attend the VIVA will result in zero marks for the assignment.

### Academic Integrity:

- Plagiarism, collusion, and academic dishonesty will not be tolerated.
- Any instances of academic misconduct will be reported to the authorities and may result in severe penalties.

# Objective

The objective of this assignment is to develop a strong practical and conceptual foundation in Natural Language Processing (NLP). Students will explore the complete NLP pipeline, starting from raw text acquisition and preprocessing to statistical and neural language modeling, subword segmentation, and modern embedding techniques.

Through hands-on implementation, students will:

- Apply regular expressions for pattern matching and information extraction
- Perform text preprocessing, normalization, tokenization, stemming, lemmatization, POS tagging, and named entity recognition
- Build and evaluate statistical language models (unigram and bigram) with smoothing and perplexity analysis
- Implement Byte Pair Encoding (BPE) for subword modeling, including Urdu Unicode text processing
- Construct and analyze vector space representations, including TF-IDF, distributional embeddings (PPMI), neural word embeddings (Skip-Gram), and contextual embeddings (BERT)
- Evaluate representations using intrinsic and extrinsic metrics, emphasizing semantic similarity and downstream task performance

By the end of this assignment, students will gain the ability to design, implement, evaluate, and critically analyze NLP models, bridging classical techniques with modern representation learning approaches used in real-world NLP systems and research.

## Part 1: Regular Expressions and Preprocessing (25%)

This part introduces the complete **text preprocessing pipeline** used in NLP: acquiring text, cleaning and normalization, regex-based pattern extraction, tokenization, stopwords handling, stemming/lemmatization, and basic linguistic annotation (POS + NER). Students will also perform **small error analysis** to develop evaluation habits.

### Tools & Required Libraries

- Nltk
- BeautifulSoup4
- Punkt
- Stopwords
- Averaged\_perceptron\_tagger
- Maxent\_ne\_chunker
- Words
- Wordnet
- omw-1.4

### Dataset / Text Source (Required)

Use one public web page as your dataset (English, non-poetry), such as:

- MIT CSAIL people page (example):  
<https://www.csail.mit.edu/people/>
- If it does not work, use any other informational webpage **URL** (university department page, organization page, research lab page).

### Constraints

- Use a webpage containing real text and ideally contact info and other textual information as well which can satisfy tasks.

- You must include the final URL used in your report.

### Task:P1T1. HTML to Plain Text Utility

Write a function: `def fetch_clean_text(url: str) -> str:`

It should:

- 1) download HTML using `urllib.request`
- 2) extract text using `BeautifulSoup(html).get_text()`
- 3) return plain text

#### Deliverable:

print the first 500 characters of cleaned text and before cleaning

### Task:P1T2. Tokenization + Regex Extraction (Emails & Phones)

Using text from P1T1:

1. Email + Phone extraction
2. Tokenization Tokenize using `nltk.word_tokenize()` and `nltk.sent_tokenize()`.

**Important:** Do NOT tokenize before extracting emails (emails break during tokenization).  
Extract using regex on the **raw cleaned text**.

#### Deliverable:

- Total sentences
- Total tokens
- List of unique emails
- List of unique phone numbers
- show your regex patterns

### Task:P1T3. Text Normalization Pipeline

Using text from P1T2:

Normalization must include:

- lowercasing
- remove extra whitespace
- optional: replace digits with `<NUM>`
- optional: remove punctuation (must justify choice)

#### Deliverable:

- show before/after snippet (first 300 chars)
- mention which rules you applied and why

### Task:P1T4. Stopwords and Corpus Statistics

Using text from P1T3:

Given a text corpus, perform the following steps:

1. Compute the top-20 most frequent tokens before stopwords removal.
2. Remove English stopwords using NLTK.
3. Compute the top-20 most frequent tokens after stopwords removal.

#### Metrics to Report

- Total token count (before and after stopwords removal)
- Vocabulary size (number of unique tokens, before and after)
- Type-Token Ratio (TTR) for both cases

$TTR = (\text{Number of unique tokens}) / (\text{Total number of tokens})$

#### Deliverables

- Two top-25 frequency lists (before vs after stopwords removal)
- A statistics summary table

### Task:P1T5. POS Tagging + POS Distribution

Using text from P1T3:

Perform POS tagging on normalized tokens:

- `nltk.pos_tag(tokens)`

#### Deliverable:

- Top 10 POS tags by frequency
- Extract 15 nouns and 15 verbs (examples)
- POS distribution table
- Example noun/verb lists

### Task:P1T6. Lemmatization vs Stemming

Using text from P1T3:

Apply the following normalization techniques to a set of 125 tokens:

- **Porter Stemmer**
- **Lancaster Stemmer**
- **WordNet Lemmatizer**

#### Deliverables

1. A **comparison table** with the following columns:
  - Token
  - Porter Stem
  - Lancaster Stem
  - Lemma

| Token | Porter | Lancaster | Lemma |
|-------|--------|-----------|-------|
|-------|--------|-----------|-------|

### Task:P1T9. Named Entity Recognition (NER)

Perform NER using NLTK chunker:

- `nltk.ne_chunk(pos_tags)`

Extract and report unique named entities under:

- PERSON
- ORGANIZATION
- GPE/LOCATION

**Deliverable:**

- Entity lists + counts

## Part 2 (Language Models and Smoothing ) 25%

This question requires you to train some language models on a training corpus and to test them on two smaller corpora. Starter code is also provided with this assignment. Each sentence must be surrounded by a start of sentence and end of sentence marker (<s> ... </s>). Preprocess to add these sentence markers. These markers will allow your models to generate sentences that have realistic beginnings and endings.

### Dataset

Brown Corpus (NLTK)

Reuters Corpus (NLTK)/Gutenberg Corpus (NLTK) **for out of Domain Test**

### Implement the following models:

1. UnigramModel: an unsmoothed unigram model
2. SmoothedUnigramModel: a unigram model smoothed using Laplace (add-one) smoothing,
3. BigramModel: an unsmoothed bigram model
4. SmoothedBigramModelLL: a bigram model smoothed using Linear Interpolation

### For each of the four language models, you need to implement the following methods:

- **generateSentence(self)**: returns a sentence sent that is generated by the language model. sent is a list of the form [<s> w1, ....., wn </s>]. You can assume that <s> starts each sentence (with probability 1). The following words (<s> w1, ....., wn </s>) are generated according to your language model's distribution. The number of words (n) is not fixed. Instead, you stop generating a sentence as soon as you generate the end of sentence symbol </s>.
- **getSentenceProbability(self, sen)**: returns the probability of the sentence sen (which is again a list of the form [<s> w1, ..., wn </s>]) according to the model. Please use the provided generateSentencesToFile method and your unigram and bigram language models to generate 20 sentences (saved as unigram\_output.txt, smoothunigram\_output.txt, bigram\_output.txt and smoothbigramLL\_output.txt).

### Perplexity

You need to compute the perplexity (normalized inverse log probability) of the two test corpora according to all of your models. Evaluate the models on the test corpora. Do you see a difference between the two test domains?

### Questions

1. When generating sentences with the unigram model, what controls the length of the generated sentences? How does this differ from the sentences produced by the bigram models?
2. Consider the probability of the generated sentences according to your models. Do your models assign drastically different probabilities to the different sets of sentences? Why do you think that is?
3. Generate additional sentences using your bigram and smoothed bigram models. In your opinion, which model produces

- better / more realistic sentences?
4. For each of the four models, which test corpus has a higher perplexity? Why? Make sure to include the perplexity values in the answer.

## Part 3 Byte Pair Encoding (25%)

The objective of this part is to provide hands-on experience with Unicode text processing using the Byte Pair Encoding (BPE) algorithm. BPE is a data-driven subword tokenization technique widely used in English NLP, where it helps reduce vocabulary size, handle rare words, and improve generalization by decomposing words into frequent subword units.

In this assignment, BPE is applied to Urdu Unicode text to understand subword segmentation in the context of a morphologically rich and low-resource language. Students will learn how BPE merges character sequences to form meaningful subwords, while correctly handling Unicode normalization, token boundaries, and script-specific characteristics of Urdu.

This part highlights how BPE enables better vocabulary coverage, reduced out-of-vocabulary (OOV) rates, and more robust representations, both in English and Urdu, and prepares students for modern embedding-based NLP models.

### Helping Material

For background on the BPE algorithm, refer to Chapter 2 of the course textbook: Speech and Language Processing (Jurafsky & Martin) <https://web.stanford.edu/~jurafsky/slp3/>

Use a good Unicode text editor (such as [BabelPad](#)) to view Urdu files. [Here](#) is a link to the Unicode Urdu code page.

### Task:P3T1 — Byte Pair Encoding (BPE) for Urdu Word Segmentation

In this task, you are provided with 50 unsegmented Urdu sentences along with a dictionary (wordlist). Your objective is to implement the Byte Pair Encoding (BPE) algorithm to segment the unsegmented Urdu text into meaningful subword units and words.

You must apply BPE-based segmentation to each sentence and generate a segmented version of the input text. You may preprocess the dictionary to simplify or optimize the lookup process; however, any preprocessing steps or assumptions must be clearly explained in no more than 100 words in your report.

#### Input Files

- `wordlist.txt` — Urdu dictionary for lookup
- `50_nospaces.txt` — 50 Urdu sentences without spaces

#### Output

- `50_segmented.txt`
  - Contains the **segmented version** of each input sentence
  - One segmented sentence per line
  - UTF-8 encoded

### Task: P3T2 — Average Characters per Word in Urdu Text

In this task, you are required to calculate the average number of characters per word in a running Urdu text. You must collect a large Urdu corpus containing at least 10,000 tokens from the internet, such as news articles or blog posts (poetry is not allowed).

After tokenizing the text, compute the number of **Unicode characters per token** and report the **average characters per token**.

### Input

- A self-collected Urdu text corpus ( $\geq 10,000$  tokens)

### Output

- Total number of tokens
- Average number of characters per token
- Source link(s) of the corpus used

All results must be clearly reported in the written report.

## PART 4: EMBEDDING MODELS(25%)

The goal of this part is to systematically transition from discrete, sparse text representations—such as n-grams, statistical language models, and subword tokenization methods (e.g., BPE)—to dense, continuous vector representations in the form of word, sentence, and contextual embeddings.

Students will implement, analyze, and compare multiple embedding paradigms, ranging from classical count-based models (TF-IDF) to neural distributional embeddings (Word2Vec, GloVe, FastText) and contextual representations derived from pretrained transformer models (e.g., BERT-style embeddings). Emphasis will be placed on understanding the theoretical foundations of representation learning, including distributional semantics, dimensionality reduction, and context sensitivity.

The effectiveness of these representations will be rigorously evaluated using both intrinsic metrics (e.g., cosine similarity, word similarity benchmarks, analogy tasks) and extrinsic evaluations through downstream NLP tasks such as text classification, clustering, and semantic search. Students will be required to analyze trade-offs in performance, interpretability, computational cost, and domain adaptability, grounding their conclusions in empirical evidence.

### Datasets

#### Primary Dataset (Unlabeled)

- **Purpose:** Representation learning
- **Source:** Wikipedia or News articles from Hugging Face
- **Size:** ~20,000–100,000 sentences
- **Domain:** General / informational
- **Preprocessing:** Regex normalization & tokenization (Part-1)
- **Tokenization variants:**
  - Word-level
  - BPE subwords (from Part-3)

#### Secondary Dataset (Labeled)

- **Purpose:** Extrinsic evaluation (downstream task)
- **Choose any one of the following:**
  - News topic classification (e.g., AG News)
  - Sentiment analysis (e.g., IMDb / product reviews)

### TASK:P4T1. TF-IDF (Baseline)

Implement from scratch using the primary dataset.

- Build vocabulary (min freq  $\geq 5$ )
- Compute TF, IDF, TF-IDF
- Generate document vectors using:
  - Word tokens
  - BPE tokens

#### Report Metrics

- Vocabulary size
- Sparsity ratio (% zeros)
- OOV rate
- Cosine similarity (document pairs)

#### Comparison

- Word-level vs BPE-based TF-IDF

### TASK:P4T2. Count-Based Word Embeddings (PPMI)

Implement distributional embeddings.

- Build word–context co-occurrence matrix
- Context windows: **k = 2, 5**
- Apply **PPMI**

#### Report Metrics

- Nearest neighbors (cosine)
- Average top-k similarity
- Vocabulary coverage

#### Analysis

- Effect of window size

### TASK:P4T3. Neural Word Embeddings (Word2Vec / SGNS)

Implement SGNS from scratch (no embedding libraries).

- Embedding sizes: **50, 100**
- Negative samples: **5–10**
- Optimizer: **SGD**
- Train on:
  - Word tokens
  - **BPE tokens**

#### Report Metrics

- Training loss & convergence
- Training time / efficiency
- OOV rate & coverage
- Average cosine similarity
- Semantic consistency (neighbors)

### Analysis

- Does BPE improve rare/OOV word representations?

### P4T4. Intrinsic Evaluation

Evaluate **semantic quality**.

- Custom word similarity set (20–30 pairs)
- **Spearman correlation**
- Nearest-neighbor coherence

### Compare

- TF-IDF
- PPMI-SVD
- SGNS (Word2Vec)

### P4T5. Sentence Embeddings

Construct sentence vectors using **static embeddings** (SGNS or GloVe-pretrained).

- Mean pooling
- TF-IDF weighted average
- **SIF (Smooth Inverse Frequency)**

### Use Case

- Sentence similarity / paraphrase detection

### Metrics

- Cosine similarity
- Accuracy (threshold-based)
- ROC-AUC (optional)

### TASK:P4T6. Extrinsic Evaluation

Use embeddings as features for **ONE** downstream task:

- Text classification / Sentiment / Topic classification / Semantic search

### Compare Representations

- TF-IDF
- Word embeddings (sentence-level)

### Report

- Accuracy
- F1-score

## **Mandatory Metrics Summary**

Students must report:

- Vocabulary size
- OOV rate
- Sparsity ratio
- Cosine similarity
- Spearman correlation
- Accuracy & F1-score (extrinsic task)

## **Deliverables**

### **1. 1 Notebook**

- A single notebook containing code implementations for all parts, organized sequentially by the specified TASK IDs.
- All results must be visible and reproducible, with every cell already executed.
- The notebook must not require re-running to verify outputs.

### **2. 1 Detailed Report**

- A well-structured written report covering each task separately.
- Include screenshots of results and outputs for every task.
- Ensure proper academic formatting, clear headings, and consistent labeling.

### **3. Exported Files**

- All required exported artifacts (e.g., generated data files, plots, models, or result files) referenced in the notebook and report.

Screenshots in the report should correspond to key results and tables shown in the notebook.

## **Important Notes**

- Incomplete execution, missing TASK IDs, or improperly formatted reports may result in loss of marks.
- Ensure consistency between notebook outputs, screenshots, and exported files.